

ASSIGNMENT 8 - AI PROGRAMMING

1. What does **image classification** mean? Explain with one example.

Image classification is the task of assigning one label to an image from a fixed set of possible classes.

The input is an array of pixel values and the output is a probability distribution over the classes, where the highest score becomes the predicted label.

Example: a model receives a 28×28 image of a handwritten digit and outputs the label "7".

2. Why is using a **Dense Network** not always a suitable choice for images?

A fully connected network ignores the local spatial structure of an image.

When a 28×28 image is flattened into a vector of 784 numbers, the model no longer knows which pixels were neighbours, so learning visual patterns becomes inefficient. It also has to learn separate weights for the same pattern at every position.

A CNN avoids this because one small filter scans the whole image and can detect the same pattern wherever it appears (left, right, top, or bottom).

3. In a CNN, what is **convolution**, and what does a filter or kernel do?

Convolution is the main operation in CNN. A filter (or kernel) is a small matrix, often 3×3 , that slides over the image. At each position it multiplies the overlapping pixel values by the filter values and adds the results, producing a single number that measures how strongly that local region matches the pattern stored in the filter. A vertical-edge filter, for example, gives a strong response where the left side is bright and the right side is dark.

4. What is a **feature map** in a convolutional neural network?

A feature map is the output produced when one filter is applied across the entire input. Each value in the map shows how strongly the corresponding local region matches the pattern that filter detects. If a layer has 32 filters, it produces 32 feature maps, one per filter.

5. Why is **weight sharing** used in CNNs? Explain.

Weight sharing means the same filter (the same set of weights) is reused at every position in the image instead of learning a separate detector for each location. This drastically reduces the number of parameters and gives translation tolerance: a useful visual feature can be detected no matter where it appears in the image.

6. How does the **ReLU activation function** work? Write the ReLU output for the following values: `[-3, 0, 2, 5]`

ReLU (Rectified Linear Unit) keeps positive values unchanged and replaces negative values with zero, which adds nonlinearity so the network can learn complex patterns.

Rule: $\text{output} = \max(0, x)$.

Applied to `[-3, 0, 2, 5] → [0, 0, 2, 5]`

7. What is **max pooling**, and why is it used in CNNs?

Max pooling slides a small window, typically 2×2 , over a feature map and keeps only the largest value in each window. It is used to reduce the spatial size of the feature maps (less computation in later layers), retain the strongest signal, and add tolerance to small shifts in object position. Example: the window `[1, 3, 2, 0]` pools to `3`.

8. In the Fashion-MNIST code, why are images reshaped from `28 × 28` to `28 × 28 × 1`?

A Conv2D layer expects each image to have height, width, and channel dimensions. Fashion-MNIST images are grayscale, so they have only one channel, and the explicit channel dimension ($\times 1$) must be added to match the format Conv2D requires. A colour image would instead be shaped $28 \times 28 \times 3$ for the red, green, and blue channels.

9. What is the difference between **training data, validation data, and test data**?

Training data is used to update the model weights.

Validation data is used during development to check whether the model is generalizing (for example, to tune settings or trigger early stopping). It does not update the weights.

Test data is held out until the very end to give an honest estimate of performance on unseen data.

10. What is **overfitting**? If training accuracy is high but validation accuracy is low, what does this indicate?

Overfitting is when a model learns the training data too specifically, including noise and irrelevant details, and therefore performs worse on new data. High training accuracy combined with low validation accuracy indicates overfitting: the model has memorized training patterns instead of learning general visual rules. Common fixes include more data, data augmentation, dropout, a simpler model, or stronger regularization.